C Practice Workbook

Medium-to-hard problems & interview prep — offline, no solutions handed to you

Active practice, not revision. Coding problems across strings, arrays, pointers & memory, bit manipulation, recursion, data structures, and file I/O — each tagged **(M)** medium or **(H)** hard, with a hint but no solution: you write them, then bring them to review. Plus a full interview section — C conceptual questions, "what does this print?" output-prediction, and find-the-bug challenges — each with an **answer key at the back** so you can grade yourself with no internet. Compile everything with `-Wall -Wextra -g` and run it under `valgrind`.

How to use this workbook

Medium-to-hard C practice, built to be worked with no internet. Problems are tagged **(M)** medium or **(H)** hard. Rules:

- **Attempt before checking.** Parts H, I, J have an answer key at the back — don't read it until you've committed to an answer.
- **Compile clean, run clean.** `gcc -Wall -Wextra -g prog.c -o prog`, then `valgrind --leak-check=full ./prog`. Zero warnings, zero leaks.
- **The coding problems (A–G) give specs and hints, not solutions.** You write them; bring the code to review.
- **Type** `man 3 <fn>` for any library function you're unsure of — that's your offline reference.

This pairs with the curriculum: do a phase there, then pull the matching problems here.

Part A — Strings (medium → hard)

- [ ] **A1 (M) Reverse words in a sentence.** `"the sky is blue"` -> `"blue is sky the"`. Stretch: in place. *Tests two-pointer + in-place.*
- [ ] **A2 (M) Anagram check.** Are two strings anagrams? O(n) with a 256-entry count array, not sorting. *Tests counting.*
- [ ] **A3 (M) First non-repeating character.** Return the first char appearing exactly once, or report none. *Tests frequency count, two passes.*
- [ ] **A4 (M) Run-length encode and decode.** `"aaabbc"` <-> `"a3b2c1"`. Do both directions. *Tests parsing + building output.*
- [ ] **A5 (H) Implement `atoi`.** Skip leading spaces, optional sign, stop at first non-digit, clamp overflow to `INT_MAX` / `INT_MIN`. *Classic interview filter.*
- [ ] **A6 (H) Implement `itoa`.** Int to string into a caller buffer; handle negatives and 0. *Tests digit extraction + reversal + sign.*
- [ ] **A7 (M) Palindrome check**, ignoring case and non-alphanumerics. `"A man, a plan, a canal: Panama"` -> true. *Tests filtered two-pointer.*
- [ ] **A8 (H) Longest substring without repeating characters** — return its length. *Sliding window; a top interview question.*
- [ ] **A9 (H) Your own `strtok`** without global state (pass a `char **saveptr`). *Tests pointers + the real strtok design flaw.*
- [ ] **A10 (H) All permutations of a string.** Print them all; stretch: skip duplicates. *Tests recursion/backtracking + swapping.*
- [ ] **A11 (M) Count words** in a string, handling multiple/leading/trailing spaces. *Tests a state machine.*
- [ ] **A12 (H) Wildcard match** with `?` (any one char) and `*` (any run). *Recursion or DP — hard.*

Part B — Arrays & two-pointer (medium → hard)

- [ ] **B1 (M) Reverse in place, then rotate left by k** using three reversals. *Tests the reversal trick.*
- [ ] **B2 (M) Remove duplicates from a sorted array** in place; return the new length. *Tests slow/fast pointer.*
- [ ] **B3 (M) Move all zeros to the end**, order preserved, in place. *Tests a write index.*
- [ ] **B4 (H) Kadane's algorithm** — maximum subarray sum. *DP intuition — classic.*
- [ ] **B5 (H) Dutch national flag** — sort an array of 0/1/2 in one pass. *Three-pointer partition.*
- [ ] **B6 (M) Two-sum on a sorted array** — return the two indices, two-pointer, O(n). *Tests two-pointer.*
- [ ] **B7 (M) Merge two sorted arrays** into one sorted array. *The merge step of mergesort.*
- [ ] **B8 (H) Majority element** (appears > n/2) in O(n) time, O(1) space — Boyer–Moore voting. *Tests a clever invariant.*
- [ ] **B9 (M) Second-largest element** in a single pass, with correct edge cases. *Tests careful bookkeeping.*
- [ ] **B10 (H) Product of array except self**, no division, O(n). *Prefix/suffix products.*

Part C — Pointers, memory & the C-specific stuff (medium → hard)

- [ ] **C1 (M) Swap via pointers**, then swap two pointers themselves through `int **`. *Tests pointer-to-pointer.*
- [ ] **C2 (M) Generic swap** — `void swap(void *a, void *b, size_t size)` via byte copy. *Tests void* + memcpy.*
- [ ] **C3 (H) Generic sort** — `sort(void *base, size_t n, size_t size, int (*cmp)(const void*, const void*))` (bubble is fine). *Tests function pointers + byte-wise pointer math — this is how `qsort` works.*

- [ ] **C4 (M) A vector of ints** — `push`, `pop`, `get`, amortized growth, clean free. *Tests realloc + ownership.*
- [ ] **C5 (H) A generic vector** storing elements of any size (`void*` + `elem_size`). *The hard version of C4.*
- [ ] **C6 (M) Deep-copy a `char **`** (array of strings) and free every piece. *Tests the char memory model.*
- [ ] **C7 (H) A bump/arena allocator** — one big `malloc`, hand out aligned slices, free all at once. *Real allocator thinking.*
- [ ] **C8 (M) Word-frequency counter** — read a file into a dynamic list of `{word, count}`. *Structs + dynamic memory + strings together.*

Part D — Bit manipulation (medium → hard)

- [ ] **D1 (M) Count set bits**, then redo with Brian Kernighan's `n & (n-1)` trick. *Tests bit loops.*
- [ ] **D2 (M) Is it a power of two?** in O(1). `n > 0 && (n & (n-1)) == 0`.
- [ ] **D3 (M) Swap two ints without a temp** (XOR). *Tests the XOR trick.*
- [ ] **D4 (H) Single number** — all elements appear twice except one; find it in O(n)/O(1) with XOR. *Tests the XOR invariant.*
- [ ] **D5 (M) Reverse the 32 bits** of an unsigned int. *Tests shift + mask.*
- [ ] **D6 (M) Get / set / clear / toggle** the i-th bit. *Tests bit masks.*
- [ ] **D7 (H) Counting bits 0..n** — build the DP where `bits[i] = bits[i >> 1] + (i & 1)`. *Tests bit DP.*
- [ ] **D8 (H) Add two ints without `+`** — bitwise sum and carry loop. *Tests adder logic.*

Part E — Recursion & backtracking (medium → hard)

- [ ] **E1 (M) Fibonacci recursively, then memoized** — and compare the call counts. *Tests recursion vs memoization.*
- [ ] **E2 (M) Tower of Hanoi** — print every move. *Classic recursion.*
- [ ] **E3 (H) Power set** — generate all subsets of an array. *Tests backtracking.*
- [ ] **E4 (H) Balanced parentheses** — generate all valid strings of n pairs. *Backtracking with constraints.*
- [ ] **E5 (H) Rat in a maze** — find a path through a 0/1 grid. *DFS/backtracking on a matrix — ties to your matrix work.*
- [ ] **E6 (M) Recursive binary search** on a sorted array. *Divide and conquer.*

Part F — Data structures from scratch (hard)

Your own structs, your own malloc/free; every one must pass valgrind with zero leaks.

- [ ] **F1 (M) Singly linked list** — insert front/back, delete by value, print, free the whole list. *Tests pointer surgery + cleanup.*
- [ ] **F2 (H) Reverse a linked list** iteratively, then recursively. *The classic.*
- [ ] **F3 (H) Detect a cycle** (Floyd's slow/fast pointers). *Two-pointer on lists.*
- [ ] **F4 (H) Merge two sorted linked lists.** *Pointer merging.*
- [ ] **F5 (M) Stack** — array-backed and list-backed versions. *LIFO, two ways.*
- [ ] **F6 (M) Queue** as a circular buffer. *FIFO + wraparound indexing.*
- [ ] **F7 (H) Hash table** with chaining — `set`/`get`/`delete`, djb2 hash, stretch: resize on load factor. *Ties straight to your KV-store project.*
- [ ] **F8 (H) Binary search tree** — insert, search, in-order traversal, and delete (including the two-children case). *Recursion + pointers.*
- [ ] **F9 (H) Min-heap** — push, pop-min, heapify, array-as-tree indexing. *Tests index math.*
- [ ] **F10 (H) LRU cache** — hash map + doubly linked list, O(1) get/put. *A very common hard interview question.*

Part G — File I/O & systems-flavored programs (medium → hard)

- [ ] **G1 (M) `wc` clone** — lines, words, bytes, with `-l`/`-w`/`-c` flags. *File I/O + arg parsing.*
- [ ] **G2 (M) `head` / `tail`** — first/last N lines (tail needs a ring buffer). *Tests buffering.*
- [ ] **G3 (H) `grep` clone** — substring match across a file with line numbers; stretch: simple patterns. *String search + I/O.*
- [ ] **G4 (H) Hex dump (`xxd`)** — offset, 16 hex bytes, ASCII gutter. *Binary I/O + formatting; CTF-useful.*
- [ ] **G5 (H) CSV parser** — handle quoted fields with embedded commas. *A real state machine.*
- [ ] **G6 (H) Persistent key-value store** — load from a file on start, save on exit. *I/O + structs + parsing.*

Part H — Interview conceptual questions (C-specific)

Answer each in a sentence or two, then check the key. These are what C interviewers actually ask.

- [ ] **H1.** Difference between `char *s = "hi";` and `char s[] = "hi";`?

- [ ] **H2.** Why does `sizeof` an array *parameter* inside a function give the pointer size?
- [ ] **H3.** What is array-to-pointer decay, and where does it NOT happen?
- [ ] **H4.** What does `static` mean on (a) a local variable, (b) a global or function?
- [ ] **H5.** Difference between `malloc` and `calloc`? When does `realloc` move the block?
- [ ] **H6.** Define: dangling pointer, wild pointer, memory leak.
- [ ] **H7.** Why is `strcpy` unsafe, and what do you use instead?
- [ ] **H8.** Difference between `memcpy` and `memmove`?
- [ ] **H9.** What is undefined behavior? Give three examples.
- [ ] **H10.** Stack vs heap — what lives where, who manages each?
- [ ] **H11.** `const char *p` vs `char *const p` — what's const in each?
- [ ] **H12.** `++i` vs `i++` — when does the difference actually matter?
- [ ] **H13.** Why must `fgetc`'s result be stored in an `int`, not a `char`?
- [ ] **H14.** What is a function pointer? Give one real use.
- [ ] **H15.** Difference between a declaration and a definition?
- [ ] **H16.** What does `volatile` do?
- [ ] **H17.** Why can comparing strings with `==` be wrong?
- [ ] **H18.** What is integer promotion? What is `sizeof('a')` in C?
- [ ] **H19.** What is a segmentation fault at the OS level?
- [ ] **H20.** How is `int a[3][4]` laid out in memory, and how does that differ from `int **`?

Part I — What does this print? (output prediction)

Reason it out, write your answer, then check the key. Some are undefined behavior — saying so is the correct answer.

I1

```c
int a[] = {1, 2, 3, 4, 5};
printf("%zu\n", sizeof(a));
printf("%zu\n", sizeof(a) / sizeof(a[0]));
```

I2

```c
char s[] = "hello";
printf("%zu %zu\n", strlen(s), sizeof(s));
```

I3

```c
int x = 5;
printf("%d %d %d\n", x++, x++, x++);
```

I4

```c
int i;
for (i = 0; i < 5; i++);
printf("%d\n", i);
```

I5

```c
char *p = "hello";
printf("%c %c %c\n", *(p + 1), p[1], *p + 1);
```

I6

```c
printf("%zu %zu\n", sizeof('a'), sizeof(char));
```

I7

```c
int arr[5] = {1, 2};
printf("%d %d %d\n", arr[2], arr[3], arr[4]);
```

I8

```c
int x = 1;
if (x = 0) printf("true\n");
else       printf("false\n");
```

I9

```c
int a[] = {10, 20, 30};
int *p = a;
printf("%d\n", *p++);
printf("%d\n", *p);
```

I10

```c
int i = 5, *p = &i, **pp = &p;
**pp = 10;
printf("%d\n", i);
```

I11

```c
void f(void) { static int c = 0; printf("%d ", ++c); }
/* f() called three times */
```

I12

```c
int a = 10, b = 3;
printf("%d %d %f\n", a / b, a % b, (float)a / b);
```

Part J — Find the bug

Each snippet has at least one real bug (memory, pointer, or logic). Name it and give the fix. Key at the back.

J1

```c
char *greet(void) {
    char msg[] = "hello";
    return msg;
}
```

J2

```c
char *s = malloc(5);
strcpy(s, "hello");
```

J3

```c
char buf[10];
strcpy(buf, "this string is far too long");
```

J4

```c
int *p;
*p = 5;
```

J5

```c
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);
```

J6

```c
char a[] = "hello", b[] = "hello";
if (a == b) printf("equal\n");
```

J7

```c
int arr[5];
for (int i = 0; i <= 5; i++)
    arr[i] = 0;
```

J8

```c
char dst[5];
strncpy(dst, "hello", 5);
printf("%s\n", dst);
```

J9

```c
int *p = malloc(sizeof(int));
free(p);
free(p);
```

J10

```c
int *make(int n) {
    int arr[n];
    for (int i = 0; i < n; i++) arr[i] = i;
    return arr;
}
```

Appendix — Answer keys

Part H — conceptual

- **H1.** `char *s = "hi"` points at a read-only string literal (modifying it is UB); `char s[] = "hi"` is a mutable local array initialized with a copy of the text.
- **H2.** An array parameter *decays* to a pointer, so inside the function the name is a pointer; `sizeof` reports the pointer's size (8), not the original array's. Pass the length separately.
- **H3.** Decay is the automatic conversion of an array name to a pointer to its first element in most expressions. It does NOT happen as the operand of `sizeof`, as the operand of `&`, or for a string literal initializing a char array.
- **H4.** (a) On a local, `static` gives it program-long lifetime that persists across calls (but still local scope). (b) On a global or function, it limits linkage to the current file (not visible to other translation units).
- **H5.** `malloc(n)` leaves memory uninitialized; `calloc(count, size)` zeroes it and does the multiplication with an overflow check. `realloc` may move the block when it can't grow in place; always assign its return to a temp.
- **H6.** Dangling = points to memory that was freed or went out of scope. Wild = never initialized, points anywhere. Leak = allocated memory with no remaining pointer to free it.
- **H7.** `strcpy` doesn't know the destination size, so a too-long source overflows it (classic buffer overflow). Use `snprintf`, or `strncpy` plus a manual `'\0'`.
- **H8.** Both copy n bytes, but `memmove` is safe when source and destination overlap; `memcpy` is undefined behavior on overlap (though often faster).
- **H9.** UB = the standard imposes no requirements; anything may happen. Examples: dereferencing NULL/freed/uninitialized pointers, out-of-bounds access, signed integer overflow, modifying a variable twice between sequence points.
- **H10.** Stack holds locals and call frames, managed automatically (freed on return); heap holds `malloc`'d memory you manage and must `free`. Stack is small and fast; heap is large and manual.
- **H11.** `const char *p` — the characters are const (can't write `*p`), but `p` can be repointed. `char *const p` — `p` is const (can't repoint), but you can modify `*p`.
- **H12.** Both increment; `++i` evaluates to the new value, `i++` to the old. It matters only when the result is used in the same expression (e.g. `a[i++]`), not in a plain statement.
- **H13.** `fgetc` returns `int` so it can represent every byte *and* a distinct negative `EOF`. Stored in a `char`, `EOF` can collide with a valid byte or sign-extend wrong, breaking the end test.
- **H14.** A variable holding the address of a function, called through the pointer. Uses: callbacks (`qsort`'s comparator), dispatch tables, plugin-style hooks.
- **H15.** A declaration announces a name and type (no storage); a definition also allocates storage / provides the body. You can declare many times, define once.
- **H16.** `volatile` tells the compiler the value may change outside normal program flow (hardware register, signal handler, another thread), so it must not cache it in a register or optimize away reads/writes.
- **H17.** On two `char *`/arrays, `==` compares addresses, not contents — two equal strings at different addresses compare unequal. Use `strcmp(a, b) == 0`.
- **H18.** Small integer types are promoted to `int` in expressions. A character constant like `'a'` has type `int` in C, so `sizeof('a')` is `sizeof(int)` (usually 4) — unlike C++, where it's 1.
- **H19.** The CPU's memory-management unit traps an access to a page the process may not touch (unmapped or protected), and the OS sends `SIGSEGV`, killing the process.
- **H20.** `int a[3][4]` is one contiguous block of 12 ints, row after row; `a[i][j]` is `*(base + i*4 + j)`, one dereference. `int **` is a pointer to an array of pointers, each pointing to a separate block — two dereferences, non-contiguous, and the two types are not interchangeable.

Part I — output prediction

- **I1.** `20` then `5` (with 4-byte `int`): whole-array size, then element count.
- **I2.** `5 6` — length excludes `'\0'`, size includes it.
- **I3. Undefined behavior** — `x` is modified multiple times with no sequence point between; argument evaluation order is unspecified too. Any output (or none) is "allowed." The lesson: never do this.
- **I4.** `5` — the trailing `;` is the loop body (empty), so the loop just runs `i` to 5.
- **I5.** `e e i` — `*(p+1)` and `p[1]` are both `'e'`; `*p + 1` is `'h' + 1` = `'i'`.
- **I6.** `4 1` — in C, `'a'` has type `int`, so `sizeof('a')` is 4; `sizeof(char)` is 1 by definition. (In C++ the first would be 1.)
- **I7.** `0 0 0` — a partial initializer zero-fills the remaining elements.
- **I8.** `false` — `x = 0` is an assignment that yields 0 (falsy). The classic `=` vs `==` bug.
- **I9.** `10` then `20` — `*p++` dereferences the old `p` (10), then advances `p`; next read is `20`.
- **I10.** `10` — writing through the pointer-to-pointer reaches `i`.
- **I11.** `1 2 3` — `static` persists `c` across the three calls.
- **I12.** `3 1 3.333333` — integer division and modulus, then a float division via the cast.

Part J — find the bug

- **J1.** Returns a pointer to the local array `msg`, which dies when the function returns (dangling pointer). Fix: return a `malloc`/`strdup` copy, or take a caller buffer, or make it `static`.
- **J2.** `"hello"` needs 6 bytes (5 + `'\0'`); `malloc(5)` is one short, so `strcpy` writes out of bounds. Fix: `malloc(strlen("hello") + 1)` (6).
- **J3.** `strcpy` into a 10-byte buffer overflows badly. Fix: `snprintf(buf, sizeof(buf), "%s", ...)` or a buffer large enough.
- **J4.** `p` is uninitialized (wild pointer); `*p = 5` writes to a random address — UB. Fix: point `p` at valid storage first (`int x; p = &x;` or `malloc`).
- **J5.** Use-after-free: `*p` is read after `free(p)`. Fix: don't use freed memory; set `p = NULL` after freeing to make misuse crash loudly.
- **J6.** `a == b` compares addresses, which differ, so it's always false. Fix: `strcmp(a, b) == 0`.
- **J7.** Off-by-one: `i <= 5` writes `arr[5]`, one past the 5-element array. Fix: `i < 5`.
- **J8.** `strncpy(dst, "hello", 5)` copies 5 bytes with no `'\0'` (source length equals n), and `dst` has no room for one, so `printf("%s")` reads past the buffer. Fix: leave room and terminate (`char dst[6]; dst[5] = '\0';`).
- **J9.** Double free — `free(p)` twice corrupts the allocator (UB). Fix: free once; set `p = NULL` after the first free.
- **J10.** Returns a pointer to a local VLA `arr`, which dies on return (same bug as J1, dressed up). Fix: `malloc(n * sizeof(int))` and return that; caller frees.